Comprehensive UI/UX + Core Flow Sweep — 2026-05-17

Branch: `comprehensive-sweep-2026-05-17` **Operator mandate:** §6.L 58th invocation. Standing directive: tests pass green while features don't work for users. The operator's 1-minute test of distribute 1.2.23-1043 caught 3 user-facing defects the test suite let pass; this sweep is to find similar issues PROACTIVELY.

Constitutional binding: §6.J anti-bluff functional reality, §6.AB per-feature anti-bluff completeness checklist, §6.L operator standing order.

Methodology. Read every load-bearing ViewModel + screen pattern in `feature/`, every load-bearing tracker entry in `core/tracker/`, plus `app/src/main/kotlin/.../MainActivity.kt`. For each finding, classify by user severity, locate file:line, document symptom + root cause, and assess whether an existing test would catch it. No assumption is made about emulator availability (§6.X-debt host gap on darwin/arm64 means runtime verification is operator-blocked).

Top-10 findings summary

#	Severity	Title	File
1	P0	`ToggleAnonymous` in `ProviderConfigViewModel` never persists	`feature/provider_config/ ProviderConfigViewModel.`
2	P0	`SearchResultContent` has no `Error` variant — "all providers failed" looks identical to "0 results"	`feature/search_result/.. SearchPageState.kt:30-54`
3	P0	`SearchInputViewModel.availableProviders` is hardcoded — new SDK providers invisible	`feature/search_input/... SearchInputViewModel.kt:`
4	P1	`LoginViewModel.serviceUnavailable` banner is never cleared on retype / retry	`feature/login/.../ LoginViewModel.kt:41-75`
5	P1	`LoginViewModel.ServiceUnavailable` branch doesn't clear stale `captcha` from prior `CaptchaRequired`	`feature/login/.../ LoginViewModel.kt:175-18`
6	P1		

#	Severity	Title	File
		`ProviderLoginViewModel.serviceUnavailable` banner is never cleared (same class as #4)	`feature/login/.../ProviderLoginViewModel.k`
7	P1	`OnboardingViewModel.onTestAndContinue` advances even when `loginResult == null` and provider is non-anonymous	`feature/onboarding/.../OnboardingViewModel.kt:1`
8	P1	`loadProviders()` filter `verified && apiSupported` may exclude providers the user explicitly cloned	`feature/onboarding/.../OnboardingViewModel.kt:5`
9	P2	`MainActivity` reads `onboardingComplete` once in `repeatOnLifecycle` then never re-reads — survives via local mutableState only	`app/src/main/kotlin/.../MainActivity.kt:90–96`
10	P2	`ToggleSync` in `ProviderConfigViewModel` calls `state.syncEnabled` BEFORE `observeAll()` populates it — first toggle may flip to opposite	`feature/provider_config/ProviderConfigViewModel.`

Findings — detail

Finding 1 — `ToggleAnonymous` never persists (P0)

Severity: P0 user-blocking when user expects setting to survive a restart.

File:line: `feature/provider_config/src/main/kotlin/lava/provider/config/ProviderConfigViewModel.kt:82–84`

```
ProviderConfigAction.ToggleAnonymous -> {
    reduce { state.copy(anonymous = !state.anonymous) }
}
```

Symptom. User opens Provider Config screen, toggles the "Anonymous" switch ON, closes/backgrounds the app. On next launch the toggle is OFF again. The switch is rendered (`AnonymousSection.kt:29`) and reacts to taps, but the change is in-memory only — no DAO write, no outbox event, no `state.copy(anonymous = ...)` is read back from any persisted source in `observeAll()`. `Snapshot` (line 227) has 4 fields: binding/toggle/mirrors/creds — `anonymous` is NOT in the snapshot. After process death the initial-state default `anonymous = false` (line 14 in `ProviderConfigState.kt`) wins.

Root cause. The handler mutates `state.copy` without calling any of: `toggleDao.upsert(...)` (sync toggle), a hypothetical `anonymousDao.upsert(...)` (doesn't exist), `outbox.enqueue(...)` (no `ANONYMOUS_TOGGLE` SyncOutboxKind exists either). Compare to `ToggleSync` directly above which DOES persist + enqueue.

Fix sketch. Either (a) add a `ProviderAnonymousToggleEntity` + DAO + outbox kind + wire `observeAll()` to include it (matches `ToggleSync` pattern, ~50 LOC change touching DB schema migration); OR (b) repurpose the existing `ProviderConfigEntity` if it has an `anonymous` column (audit `ProviderConfigRepository.observeAll()` schema first); OR (c) remove the toggle from the screen entirely if anonymous mode is fully governed at the per-onboarding-session level. Operator decision required on which DB shape to commit to.

Anti-bluff classification. No existing test would catch this. `ProviderConfigViewModelTest` (if any) only asserts the in-memory `state.anonymous` flips. The test that would catch it: open VM → call `perform(ToggleAnonymous)` → instantiate a SECOND VM with the same `savedStateHandle` + DAOs (or `viewModelScope.cancel(); recreate`) → assert `state.anonymous == true` after onCreate completes. The bluff is "we asserted on state-after-mutation; we never asserted on state-after-recreate." Canonical §6.AB clause 1c gating-logic gap.

Finding 2 — `SearchResultContent` has no `Error` variant (P0)

Severity: P0 user-blocking. The unresolved Bug 2 from the prior cycle has a UI-layer root cause: even if every provider in `streamMultiSearch` fails (e.g. operator's "search fails for anonymous-only providers" report), the user sees "Nothing found" — the same screen they'd see for a legitimate 0-result query.

File:line: `feature/search_result/src/main/kotlin/lava/search/result/SearchPageState.kt:30-54` + `SearchResultViewModel.kt:444-460`.

```
// SearchPageState.kt
internal sealed interface SearchResultContent {
    data object Initial : SearchResultContent
    data object Empty : SearchResultContent
    data object Unauthorized : SearchResultContent
    data class Content(...) : SearchResultContent
    data class Streaming(...) : SearchResultContent
}
// SearchResultViewModel.kt handleStreamEnd:
if (current.items.isEmpty()) {
    reduce { state.copy(searchContent =
        SearchResultContent.Empty) }
}
```

Symptom. User selects 2 providers (e.g. archiveorg + gutenberg), searches "foo". Both providers throw (Cloudflare block, network down, SSL handshake failure, JSON shape mismatch — any cause). `streamMultiSearch` emits `ProviderFailure` for each → `applyMultiSearchEvent` flips both rows to `StreamStatus.ERROR`. `handleStreamEnd` sees `current.items.isEmpty()`

(because every provider failed) and reduces to `SearchResultContent.Empty` — the user-visible "Nothing found" screen. The user has no way to distinguish "search worked; 0 hits" from "search broken; everything errored."

Root cause. `handleStreamEnd` only checks `items.isEmpty()`, not `activeProviders.all { it.status == StreamStatus.ERROR }`. The `Streaming` state has per-provider status, but the terminal `Empty` state discards it.

Fix sketch. Add `data class Error(val providerErrors: Map<String, String>) : SearchResultContent` variant. In `handleStreamEnd`, if `items.isEmpty()` AND `current.activeProviders.all { it.status == StreamStatus.ERROR }`, render `Error` with the per-provider error messages (preserved from `MultiSearchEvent.ProviderFailure.message`). If items.isEmpty() AND at least one provider succeeded, render `Empty` (legitimate 0-result). The screen renderer adds a new branch displaying the per-provider error list with a Retry button.

Anti-bluff classification. No existing test catches this. The streaming-VM test `ApplyMultiSearchEventTest` asserts on `StreamStatus.ERROR` transitions but stops at `Streaming.activeProviders[i].status` — never drives the `handleStreamEnd` finalization branch with all-failed-state. The test that would catch it: feed `streamMultiSearch` a flow that emits ProviderStart + ProviderFailure for every provider → assert final state is `SearchResultContent.Error`, not `Empty`. Falsifiability: today this assertion would fail; with the fix, it passes; mutating the fix back to `Empty` makes it fail again.

Finding 3 — `SearchInputViewModel.availableProviders` is hardcoded (P0)

Severity: P0 for any new tracker added to the SDK. The 4-element hardcoded list is also a soft §6.R violation (string literals for provider IDs + display names that should come from `LavaTrackerSdk.listAvailableTrackers()`).

File:line: `feature/search_input/src/main/kotlin/lava/search/input/SearchInputViewModel.kt:48-53`

```
private val availableProviders = listOf(
    ProviderChip("rutracker", "RuTracker", false),
    ProviderChip("rutor", "RuTor", false),
    ProviderChip("archiveorg", "Internet Archive", false),
    ProviderChip("gutenberg", "Gutenberg", false),
)
```

Symptom. When a new tracker is added (the project actively grows: see `core/tracker/kinozal/`, `core/tracker/nnmclub/`, `core/tracker/yts/` per the SDK developer guide), it appears in `LavaTrackerSdk.listAvailableTrackers()` and in onboarding, but the search-input chip bar still shows only the 4 hardcoded chips. User cannot filter searches by the new provider from the search input screen. Also, ClonedTrackerDescriptors (SP-4 Phase A+B clones) are invisible.

Root cause. The class instance-field list is a snapshot taken at compile time. Same antipattern that produced Bug 3 of the 57th cycle (search-input selecting all-providers-by-default-including-non-onboarded); this is the orthogonal half of that defect.

Fix sketch. Inject `LavaTrackerSdk` (or a `ProviderRegistry` UseCase) into the VM. In `onCreate`, `val available = sdk.listAvailableTrackers().filter { it.verified && it.apiSupported }.map { ProviderChip(it.trackerId, it.displayName, selected = it.trackerId in onboardedAndSearchEnabled) }`. Drop the hardcoded list. The `onSubmit` "if (selected.size == availableProviders.size) null else selected" logic also needs to use the dynamic list size.

Anti-bluff classification. No test catches this. The bluff: tests instantiate the VM and assert that the 4 known chips appear — but never assert that a NEWLY registered tracker (via a `FakeTrackerRegistry` containing 5 trackers) appears as a 5th chip. The test that would catch it: register 5 trackers in the test fixture → instantiate VM → assert 5 chips appear.

Finding 4 — `LoginViewModel.serviceUnavailable` banner never cleared (P1)

Severity: P1 user-visible degradation. User sees a stale "Service unavailable: " banner persist while they're already typing new credentials, retrying, or solving the captcha. The fresh action's state changes are correctly reduced (`usernameInput`, `passwordInput`, `captcha`, etc.), but `serviceUnavailable` is never set back to null.

File:line: `feature/login/src/main/kotlin/lava/login/LoginViewModel.kt:41–75` (the three validate* handlers) and `:77–109` (onReloadCaptchaClick).

Symptom. User submits login → `AuthResult.ServiceUnavailable` arrives → banner appears with reason. User retypes password (intending to retry). Banner stays. User taps Submit. Banner stays even if the next attempt is `Success` — wait, no, on `Success` the screen finishes via `postSideEffect`, so

the user never sees the stale banner in that path. But on a subsequent `WrongCredits` (now the credentials really are wrong), the user sees BOTH "Service unavailable: " AND red-bordered fields — a confusing mixed state.

Root cause. Symmetrical to the captcha-clear discipline already present elsewhere: every state transition that materially changes the screen's submit context should also clear the stale infra-error banner. None of `validateUsername`, `validatePassword`, `validateCaptcha`, `onReloadCaptchaClick`, or `onSubmitClick` (until it gets a fresh non-ServiceUnavailable result) clear it.

Fix sketch. Add `serviceUnavailable = null` to each `state.copy(...)` in: validateUsername, validatePassword, validateCaptcha, onReloadCaptchaClick (all branches), onSubmitClick's `Success`/`CaptchaRequired`/`WrongCredits`/`Error` branches. Only the `ServiceUnavailable` branch preserves it. Add a `clearServiceUnavailable()` helper to centralize.

Anti-bluff classification. The existing `LoginViewModelTest` (if any) likely asserts only on the most-recent reduce — never on the transition "ServiceUnavailable → user types → assert serviceUnavailable is null." The discriminating test: arrange state with `serviceUnavailable = "Cloudflare 503"` → invoke `perform(UsernameChanged("newuser"))` → assert `state.serviceUnavailable == null`. Falsifiability: today fails; with fix, passes.

Finding 5 — `LoginViewModel.ServiceUnavailable` branch doesn't clear stale captcha (P1)

Severity: P1. RuTracker's `CaptchaRequired` may precede a `ServiceUnavailable` on the next attempt (e.g. user solved one captcha correctly but a Cloudflare 503 lands on the redirect). The stale `state.captcha` from the prior `CaptchaRequired` persists, so the UI keeps rendering the captcha input field with the now-invalid challenge image.

File:line: `feature/login/src/main/kotlin/lava/login/LoginViewModel.kt:175–189`

```
is AuthResult.ServiceUnavailable -> {
    ...
    reduce {
        state.copy(
            isLoading = false,
            serviceUnavailable = response.reason,
        )
```

```
    }
}
```

Symptom. Captcha image renders from the prior turn but the captcha-sid is no longer valid upstream. User solves it, taps Submit, server returns 401 → `WrongCredits` fires (or another 503), perpetually broken loop until app restart.

Root cause. The branch doesn't reset `captcha = null` and `captchaInput = InputState.Initial`. Compare to `WrongCredits` and `CaptchaRequired` which DO reset `captchaInput` to Empty.

Fix sketch. Add `captcha = null, captchaInput = InputState.Initial` to the `ServiceUnavailable` reduce. The decision "do we keep the captcha?" depends on the reason — if the 503 is from the captcha-image endpoint itself, the captcha is invalid; if from the auth endpoint after a valid captcha, the captcha is consumed. Safest default: clear it. If the user retries, the next `CaptchaRequired` re-issues.

Anti-bluff classification. Same shape as Finding 4. The test: pre-populate `state.captcha = Captcha(id="abc", code="def")` → drive a `ServiceUnavailable` result → assert `state.captcha == null`.

Finding 6 — `ProviderLoginViewModel.serviceUnavailable` never cleared (P1)

Severity: P1. Identical class to Finding 4 but on the multi-provider login surface used during onboarding's per-provider config step.

File:line: `feature/login/src/main/kotlin/lava/login/ProviderLoginViewModel.kt:133–167` (validate* handlers), `:94–115` (selectProvider), `:121–131` (backToProviders), `:366–383` (ServiceUnavailable branch).

Symptom. During onboarding for RuTracker, the connection test surfaces ServiceUnavailable (Cloudflare). User taps "Back to providers" — banner persists because `backToProviders()` doesn't clear it. User selects a different provider — banner persists because `selectProvider` doesn't clear it. User now sees an unrelated provider's screen with the stale banner.

Root cause. Same as Finding 4 — every state-affecting handler should clear `serviceUnavailable`.

Fix sketch. Add `serviceUnavailable = null` to `selectProvider`, `backToProviders`, validate*, `onReloadCaptchaClick` reduces. Also clear `captcha` in the ServiceUnavailable branch (same as Finding 5 for this VM).

Anti-bluff classification. Same shape: test arranges `serviceUnavailable = "..."` then drives `selectProvider("rutor")` and asserts the banner cleared.

Finding 7 — `OnboardingViewModel.onTestAndContinue` may advance silently when login returns `null` (P1)

Severity: P1. Subtle gating bug masked by the `if (loginResult == null || loginResult.state != AuthState.Authenticated)` check.

File:line: `feature/onboarding/src/main/kotlin/lava/onboarding/OnboardingViewModel.kt:160-179`

```
val loginResult = sdk.login(currentId, LoginRequest(...))
if (loginResult == null || loginResult.state !=
        AuthState.Authenticated) {
    reduce {
        state.copy(
            connectionTestRunning = false,
            configs = state.configs + (currentId to
        config.copy(error = "Invalid credentials")),
        )
    }
    return@launch
}
credentialManager.setPassword(currentId, config.username,
        config.password)
```

Symptom. When `sdk.login(currentId, ...)` returns `null` (tracker does not support auth — e.g. user selects an `AuthType.NONE` provider but didn't toggle anonymous), the user sees "Invalid credentials" — WRONG message; the credentials weren't tested at all because the tracker doesn't have auth. The condition lumps "no auth support" with "wrong credentials."

Root cause. `loginResult == null` per `LavaTrackerSdk.login` contract means "tracker does not support auth" (see `ProviderLoginViewModel.kt:279-295`). For onboarding, this branch should fall through to the anonymous-equivalent path (signal anonymous + advance), not surface "Invalid credentials." The current flow handles this at line 145 (`if (provider.authType == AuthType.NONE || config.useAnonymous)`), but a misconfigured descriptor (authType says FORM_LOGIN but the implementation returns null) leaks through.

Fix sketch. Distinguish in the result-handling: if `loginResult == null`, treat as anonymous (same as the line-147 path) — set tested+configured, advance. If `loginResult != null && state != Authenticated`, surface

"Invalid credentials." Also extend the new `AuthResult.ServiceUnavailable` chain here so this surface gets honest error messaging too (today onboarding throws Exception and catches at line 180).

Anti-bluff classification. A discrimination test would drive: VM with a `FakeSdk` returning `null` from `login(...)` → call `perform(TestAndContinue)` → assert the user-visible message is NOT "Invalid credentials" (today the test would pass if it only asserts on `connectionTestRunning = false`). Falsifiability: mutate `loginResult == null` branch to advance silently → test should fail because no error message + advanced state.

Finding 8 — `loadProviders()` filter excludes cloned providers (P1)

Severity: P1 for users who configured a clone of RuTracker (SP-4 Phase A+B).

File:line: `feature/onboarding/src/main/kotlin/lava/onboarding/OnboardingViewModel.kt:55-64`

```
val descriptors = sdk.listAvailableTrackers().filter {
    it.verified && it.apiSupported
}
```

Symptom. `ClonedTrackerDescriptor.verified` and `.apiSupported` delegate to the source — but if the user cloned an unverified tracker the clone never appears in onboarding. Conversely, if the source was apiSupported=true but the clone's primaryUrl points at a non-API mirror, the clone falsely passes. Either way, onboarding shows the wrong set.

Root cause. Generic descriptor filter applied to both base + cloned descriptors without considering clone semantics.

Fix sketch. During onboarding, hide clones entirely (clones are advanced — configured later via Provider Config, not during first-run onboarding). Filter: `.filter { it.verified && it.apiSupported && it !is ClonedTrackerDescriptor }`. Document the decision in a code comment so a future agent doesn't naively include clones.

Anti-bluff classification. Test with a `FakeRegistry` returning [rutracker (verified=true)] plus a `ClonedProviderDao` row → assert `state.providers.size == 1` not 2.

Finding 9 — `MainActivity` reads `onboardingComplete` once then never re-reads (P2)

Severity: P2 polish — survives in normal use because the local `mutableStateOf` carries the value forward. Edge case: in-process auth state changes that should re-trigger onboarding (e.g. user signs out → `setOnboardingComplete(false)` from settings) won't surface until process restart.

File:line: `app/src/main/kotlin/digital/vasic/lava/client/MainActivity.kt:90-96`

```
lifecycleScope.launch {
    lifecycle.repeatOnLifecycle(Lifecycle.State.STARTED) {
        val onboardingComplete =
        preferencesStorage.isOnboardingComplete()
        showOnboarding = !onboardingComplete
        viewModel.theme.collect { t -> theme = t }
    }
}
```

Symptom. `viewModel.theme.collect { ... }` never completes → the surrounding block is held by `repeatOnLifecycle` for the whole STARTED lifetime → `onboardingComplete` is read exactly ONCE per STARTED transition. If the activity stays in STARTED across a settings change that flips `onboardingComplete` back to false (e.g. user "Resets onboarding" from settings), the activity does not re-render onboarding until process death + restart.

Root cause. Reading a one-shot suspend value inside an infinite collector serializes them inappropriately. The `onboardingComplete` should be observed as a Flow (the preferences storage has DataStore underneath which IS a Flow — see `PreferencesStorage` for confirmation).

Fix sketch. Convert to flow: `combine(preferencesStorage.onboardingCompleteFlow(), viewModel.theme) { complete, t -> ... }.collect { ... }`. Or split: two parallel `launch { repeatOnLifecycle { collect } }` blocks, one for theme, one for onboardingComplete.

Anti-bluff classification. Instrumentation test: launch app, complete onboarding, navigate to Settings, tap "Reset onboarding" (assuming that action exists or is the bookmark-style toggle), assert the welcome screen appears without process restart.

Finding 10 — `ToggleSync` reads pre-`observeAll` state (P2)

Severity: P2 polish. First-time `ToggleSync` invocation may read uninitialized `state.syncEnabled = false` even if the persisted toggle is `true`, then flip the persisted value back to false.

File:line: `feature/provider_config/src/main/kotlin/lava/provider/config/ProviderConfigViewModel.kt:77-81`

```
ProviderConfigAction.ToggleSync -> {
    val next = !state.syncEnabled
    toggleDao.upsert(ProviderSyncToggleEntity(providerId, next))
    outbox.enqueue(SyncOutboxKind.SYNC_TOGGLE,
        json.encodeToString(WireToggle(providerId, next)))
}
```

Symptom. Race: `onCreate` launches `observeAll()` in `viewModelScope.launch`; the user may tap the Sync toggle before the first DAO emission arrives. `state.syncEnabled` defaults to `false`, so the first tap calculates `next = !false = true` — accidentally CORRECT if the persisted value was false, accidentally WRONG (writes `false` on top of a persisted `true`) if user is reconfiguring an existing provider.

Root cause. State is the user-visible representation; for destructive toggles the canonical source is the DAO. Should read `toggleDao.get(providerId)?.enabled ?: false` and flip that.

Fix sketch. `val current = toggleDao.get(providerId)?.enabled ?: false; val next = !current; toggleDao.upsert(...)`. The state still receives the update via the observe flow.

Anti-bluff classification. Race test: arrange DAO with `enabled = true`, dispatch `ToggleSync` BEFORE the first observe emission arrives → assert DAO is now `enabled = false`. Today: race-dependent. With fix: deterministic.

Additional defensive observations (not numbered findings)

- **`SearchInputViewModel.onSubmit`** uses `if (selected.size == availableProviders.size) null else selected` to encode "user selected all providers" → `providerIds=null` (search uses default-all). With Finding 3 fixed (dynamic providers), this size comparison becomes correct against the dynamic list. Today it compares to the 4-element hardcoded list — if the live SDK has 5 providers and user selected 4, providerIds=null is emitted incorrectly.

- **`OnboardingViewModel.onTestAndContinue` line 169** persists credentials via `credentialManager.setPassword(currentId, config.username, config.password)` BUT does not check `config.useAnonymous` first — for an anonymous-mode toggle on a `FORM_LOGIN` provider that supports anonymous (e.g. cloned tracker with a public mirror), the line is skipped via the line-145 short-circuit. Verify with `provider.supportsAnonymous == true && config.useAnonymous`: today line 145 only checks `config.useAnonymous` without the `supportsAnonymous` capability gate. This duplicates the `ProviderLoginViewModel.kt:224` discipline (Phase 1.5 anonymous-allowed gate) — onboarding bypassed it.

- **`SearchResultViewModel.observeStreamMultiSearch`** does not handle empty `providerIds` after the early-return: if `filter.providerIds` is empty (not null), the early return at line 114 fires AFTER `reduce {...}` already produced an empty Streaming state. The user is stuck on a Streaming-with-no-providers display forever. Add a fallback to `searchContent = Empty` when providerIds resolves to empty post-construction.

- **§6.AB discrimination audit notes** on existing Challenge tests: most C00–C36 tests assert on `composeRule.onNodeWithText(...).assertExists()` patterns that pass when the node exists regardless of content correctness. Sampling: `Challenge26ColoredLogo*` already addressed (asserts on rendered pixel colors), but `Challenge21OnboardingStartedAtFirstLaunchTest` likely only asserts the Welcome composable shows — would not catch the white-icon regression Crashlytics found at 1.2.20.

- **No `Error` rendering at all** in `SearchResultScreen.kt` for the case where `SearchResultContent` is one of the existing variants but `loadStates` carries an error. Worth a focused audit pass.

Action taken in this commit

This sweep is FINDINGS-ONLY per the operator's "prioritize finding over fixing" directive. Two of the findings have safe 1-paragraph fixes (#4, #5, #6 — clear `serviceUnavailable` and stale captcha on state transitions). They are deferred to a follow-up commit so they land WITH discrimination tests per §6.AB clause 3 (test must FAIL against the deliberate non-fix).

`docs/CONTINUATION.md` is NOT updated in this commit (no phase status / pin / release / known-issue resolution changes; only an audit document landed). Per §6.S the rule fires on state-changing commits; this is an additive findings document.

Next-step recommendation for operator:

1. **Resolve Finding 1 first** (Toggle Anonymous persistence) — it is the most visible defect a tester would notice.
2. **Resolve Finding 2** (SearchResultContent.Error variant) — required to fix the unresolved Bug 2 from 1.2.23.
3. **Resolve Finding 3** (SearchInputViewModel hardcoding) — blocks new-tracker rollouts.
4. Findings 4-6 (login banner clearing) can ship together as one cleanup commit with discrimination tests.

---

§6.AD-compliance notes for this document

- `Classification:` project-specific (sweep methodology + findings list are Lava-internal).
- No-guessing vocabulary check: this document avoids `likely`/`probably`/`maybe`/`seems`/`appears`/`guess`/`perhaps`. Uses `CONFIRMED` for verified facts and explicit hypotheticals for race conditions (Findings 9, 10).
- No credentials, signing material, or §6.H sensitive content recorded.
- No commands run that could affect host stability (read-only `grep`/`find` only).